

Nobody prescribes before examining. What software engineering is

Everyone in this room can already program. Nobody in this room has yet built a system that outlives the person who wrote it. Today we look at nine real programmes — consumer products, aircraft, an airline, a 500-line script — and extract the discipline that separates the ones that survived from the ones that cost billions.

Unit 1 · Foundations and Process

80 minutes

No code today

Team formation at the end

9

DOCUMENTED
PROGRAMMES

\$60bn+

IN AVOIDABLE COST

470

LIVES LOST IN TWO
OF THEM

0

CAUSED BY BAD
CODE

Outcomes

What engineering means

The evidence

The discipline

Life cycle

Where the money goes

Worked example

Exercise

Homework

Terms & reading

LEARNING OUTCOMES

What you will be able to do at 2:20 PM

A student who has done the assigned pre-reading will be able to:

1

- L0 1.1** Distinguish programming from software engineering using at least three concrete criteria — time, scale, and consequence.
- L0 1.2** Explain the origin and the continuing relevance of the term *software crisis*, citing at least two documented project failures.
- L0 1.3** Describe the five classical phases of the software life cycle and name the principal artifact produced by each.
- L0 1.4** Classify a given system as a generic product, a bespoke system, an embedded system, or a data-intensive system, and justify the classification.
- L0 1.5** State the four cost drivers that make maintenance, not construction, the dominant expense in most software systems.
- L0 1.6** Diagnose a documented engineering failure by separating its *proximate technical cause* from the *process failure* that let the cause survive to production.

PART 1

What “engineering” actually means

Before we define software engineering, we define engineering — using a discipline that has been accountable for its decisions far longer than we have: medicine.

2

Everyone in this room can already treat a headache. Given a pharmacy, a search engine and enough confidence, you could hand someone a tablet that makes the pain go away. Nobody would call that medicine, and the reason has nothing to do with whether the tablet worked.

Engineering is what happens **before** the treatment, and what makes the treatment accountable afterwards. It is the discipline of committing resources to a design whose consequences you have predicted in advance, under constraints you did not choose, for people who will not accept excuses. Three things distinguish it from skilled making.

MAKING SOMETHING

A gifted healer may cure one patient better than a hospital can.

ENGINEERING SOMETHING

What the healer cannot do is guarantee the ten-thousandth patient, or explain a decision to a coroner four years later.

1.1 Engineering predicts failure before it happens

A clinician does not wait for the heart attack to discover whether the patient was at risk. She measures, scores the risk, treats, then follows up to *confirm what she already believes*. When a treatment fails at follow-up, that is expensive but survivable. When it fails because nobody thought to ask the question, that is an inquest.

Software has the same two categories. A test that fails is engineering working. A production outage caused by a case nobody enumerated is engineering that did not happen.

1.2 Engineering makes decisions traceable to reasons

Every line on a prescription can be traced to a finding, a guideline, or a decision someone signed. This is not bureaucracy. It is what makes it possible, five years later, to change one thing without silently breaking three others — and to find out who knew what, when a court asks.

1.3 Engineering accepts that the constraints are not negotiable

Dosage limits, drug interactions, consent law, what the pharmacy actually stocks, the hours the nurse can work, what the patient can afford. None of these were chosen by the clinician, and none can be argued with. The medicine consists precisely of finding a treatment that satisfies all of them at once.

Programming is producing behaviour that works. Engineering is producing behaviour that **keeps** working, under constraints you did not choose, for people who will not forgive you, in a way that someone else can change after you have gone.

Moved to reading. "Before the first prescription" — the market study, the business case, external constraints and the design freeze — is assigned as pre-reading for Session 3; it is on the course site.

PART 2

Nine programmes: one that worked, eight that did not

4

Every figure below can be checked against the sources at the end. Where reported totals differ between sources, the more conservative number is used.

THE ONE THAT WORKED

Dacia Logan — designing to a price, not to a wish

Renault · 1999–present

Commercial success

€5,000	1.8 m	6.5 m	~€20 m
target price, set first	Logans sold by 2019	Dacia customers in 15 years	saved by digital prototyping

Renault bought Dacia in 1999 with an unusual brief: build a modern, reliable three-box saloon that sells for about five thousand euros in Central and Eastern Europe. The price came *first* and everything else was negotiated against it — a method Renault calls design-to-cost. Proven, already-amortised Renault parts were used wherever a customer could not tell the difference; digital simulation replaced many physical prototypes.

The target was not fully met — the Logan launched in Romania nearer €6,280, and around €7,200 in Germany with added safety equipment. It did not matter. The car was aimed at a real buyer whose alternative was a used vehicle, and it was engineered to reach him.

FOR YOUR PROJECT

A constraint stated first, and treated as non-negotiable, produces better engineering than an open-ended wish list. Fix the constraint that actually binds — for SANTE-BASSAM that is intermittent power and unreliable connectivity — and design everything else against it.

FOUR THAT FAILED ON THE GROUND

Ford Edsel — the answer to a question the market stopped asking

1957–1959

Withdrawn after 3 years

\$250m+

lost (≈\$2.7bn today)

~118,000

cars built in three model years

2 years

between the research and the launch

Ford entered the medium-price segment with extensive market research, a new brand, and a heavily promoted launch. The research had been done for the market of 1955; the car arrived in 1957, into a recession that hit medium-priced cars hardest. Nothing here was a build failure. Ford built what it set out to build. **The requirement itself had expired.**

FOR YOUR PROJECT

Requirements have a shelf life. A specification agreed eighteen months ago and never revisited is a historical document, not a plan — which is why validation is a separate activity from verification.

Tata Nano — engineered to the target, positioned into the ground

2009–2018

Discontinued

₹1 lakh

the promise
(~\$2,000)

~300,000

built in ten years

518

sold in the whole of
2018

1

unit produced in
June 2018

The Nano hit an extraordinary engineering target: a genuinely new, genuinely cheap car, intended to move families off motorcycles. Then it was marketed relentlessly as *the cheapest car in the world* — in a market where buying a car is a public statement about having arrived.

Tata solved the stated problem brilliantly and discovered that the stated problem was not the customer's problem. The customer did not want the cheapest car. He wanted the least expensive car he would not be embarrassed to park outside his house.

FOR YOUR PROJECT

Elicitation is not transcription. What a stakeholder asks for and what would actually satisfy them diverge constantly — and no amount of construction quality closes that gap.

GM ignition switch — a specification that was never met, and nobody stopped it

2002–2015

124 deaths · \$900m forfeited

\$0.57	2.6 m	124	~10 yrs
cost of the part	cars in the first recall	deaths compensated	between approval and recall

In 2002 a GM engineer approved an ignition switch design although the supplier had stated it did not meet GM's own torque specification. Cars with the switch could be knocked out of the run position while moving, cutting the engine and disabling power steering, anti-lock brakes and — the lethal part — the airbags.

The Valukas investigation commissioned by GM found no grand conspiracy. It found something worse for our purposes: a change made quietly, records not updated, a defect category that let a safety-critical failure be logged as a customer-convenience issue, and no one person who owned the question.

FOR YOUR PROJECT

A requirement that is waived without a record is a defect with a delayed fuse. This is what traceability and change control are for — not paperwork, but the ability to answer *who accepted this, and on what evidence?*

Volkswagen — software written to satisfy the test instead of the requirement

2008–2015, litigated since

Costs above €33bn

11 m

vehicles with the
defeat device

€33bn+

cost to date

40×

the legal NOx limit,
on the road

4

executives
sentenced in 2025

Volkswagen's engine software detected the conditions of an emissions test and ran the engine clean; on the road it did not. Note precisely what was engineered here. **The system passed every test it was given.** The tests were the specification, and the specification was met — by a program written to recognise when it was being watched.

FOR YOUR PROJECT

Passing the tests is not the goal; the tests are evidence about the goal. When a team starts optimising for the measurement rather than the thing measured, that is an ethics failure long before it is a technical one.

Airbus A380 wiring — two teams, one aircraft, two versions of the design tool

2005–2007

≈2-year delay · ~\$6bn

530 km	~100,000	40,000	~2 yrs
of cable per aircraft	wires affected	connectors	delay to first delivery

Design teams in Germany and Spain worked in one version of the CATIA design system; teams in France and Britain had moved to the next version, which was a rewrite rather than an increment. Harness geometry did not survive the transfer between them. The wiring did not fit the airframe, and the work had to be redone from the design stage.

No individual engineer made an error of judgement about wiring. The failure was that the same design existed in two incompatible representations, and no process detected the divergence until physical parts refused to mate.

FOR YOUR PROJECT

This is a configuration-management failure at industrial scale — the same class of failure as two developers building against different versions of an interface. Session 24 exists because of it.

THE SAME LESSON AT ALTITUDE

Aviation is the harder case, and the more honest one, because in aviation the consequences of a bad requirement are not measured in money alone. Three of the four cases below are software failures outright.

TWO QUESTIONS, FOUR OUTCOMES

VALIDATION — did we build the right thing?

yes ↑ · no ↓

Right thing, built badly Denver baggage Southwest 2022 <i>Expensive. Recoverable.</i>	Right thing, built well Dacia Logan Boeing 787 <i>The only quadrant that pays.</i>
Wrong thing, built badly Mercifully rare — these get cancelled early	Wrong thing, built well Edsel · Nano · A380 MCAS · VW defeat device <i>No amount of testing saves these.</i>

VERIFICATION — did we build the thing right?

← no · yes →

The lower-right quadrant holds the most expensive failures in this document. They were not built badly — they were built well and aimed wrongly, which is why no amount of testing could have saved them.

A380 versus 787 — the same decade, two forecasts of how people fly

2000–2021

Engineering triumph, commercial failure

251	~700+	\$25bn+	2021
A380s built	the programme needed	development cost	production ended

Airbus read the future as hub-to-hub: congested mega-airports, scarce landing slots, therefore bigger aircraft. Boeing read it as point-to-point: passengers preferring direct flights on smaller, efficient twin-engine aircraft. Airbus built the A380 and Boeing built the 787.

The A380 is a magnificent machine — quiet, enormous, technically ahead of its rivals — and it was built to serve a market structure that airlines did not adopt at the scale required. Emirates bought nearly half of them.

FOR YOUR PROJECT

Architecture encodes a prediction about the world. When the prediction is wrong, no amount of engineering excellence inside the architecture rescues it — which is why Session 12 insists that architectural drivers be written down and revisited.

Boeing 737 MAX / MCAS — a requirement that was met, by a system that should not have existed in that form

2018–2020

346 deaths · \$20bn+

1	346	20 mo	\$2.5bn
angle-of-attack sensor feeding MCAS	people killed in two crashes	global grounding	DOJ settlement

Larger engines mounted further forward changed the aircraft's handling. Rather than redesign the airframe, Boeing added software — MCAS — to make the MAX handle like earlier 737s. There was a commercial requirement behind it that was explicit and rational: airlines must not need expensive new pilot type-training. **MCAS met that requirement.**

It also read from a single angle-of-attack sensor with no cross-check, had authority to trim the aircraft nose-down repeatedly, and was omitted from pilot manuals. A disagree alert between the two sensors was an optional extra. When one sensor failed, the system acted on false data and the crews could not overcome it. Lion Air 610 killed 189 people; Ethiopian 302 killed 157.

FOR YOUR PROJECT

Two lessons, and take both. A single point of failure in a safety-critical input is a design defect regardless of how reliable that input usually is. And a system whose existence is hidden from its operators has moved the failure out of engineering and into ethics — Session 28.

Denver International Airport baggage system — scope, schedule, and no way back

1991–1995, abandoned 2005

16 months late · ~\$560m over

16 mo	\$1.1 m	~\$560m	1 of 3
delay to airport opening	per day in holding costs	cost overrun	concourses ever served

Denver committed to the largest automated baggage system ever attempted: thousands of telecars, miles of track, dozens of networked computers. Midway through, the scope expanded from one concourse for one airline to three concourses and twenty airlines — without re-baselining cost or schedule. Testing was compressed by political pressure to open. Public demonstrations shredded and misrouted luggage.

What finally opened the airport was a conventional manual system, built in haste, alongside an automated one that served a single concourse and outbound flights only. It was abandoned entirely in 2005.

FOR YOUR PROJECT

Three failures you will be tempted to repeat: scope changed without re-planning, testing treated as the schedule's shock absorber, and no fallback path. Incremental delivery exists precisely because big-bang integration of an unprecedented system is not a plan.

Southwest Airlines, December 2022 — technical debt collecting its interest

software from 2004, meltdown in 2022

\$1.1bn and a record fine

16,700+	2 m+	\$140m	~300
flights cancelled in ten days	passengers stranded	record DOT penalty	schedule changes per batch the tool could handle

A winter storm disrupted operations across the United States. Every airline suffered; Southwest collapsed. Its crew-scheduling system, SkySolver, dated from 2004 and was built for an operation roughly two-thirds the size of the one it was running. It could not process reassignments at the volume the storm produced, so schedulers reverted to doing it by hand while crews and aircraft sat in the wrong cities. Internal warnings had been raised years earlier and modernisation deferred to protect costs.

FOR YOUR PROJECT

Technical debt is not a metaphor for untidy code. It is a real liability with a real interest rate, and it is paid in one lump sum on the worst day of the year. Session 26.

So what is *engineering* about it?

Look at what these cases have in common. Not one of them was caused by a programmer who could not program, or a nurse who could not give an injection. They were caused by:

1

A requirement nobody validated.

The Edsel and the Nano were built exactly as specified, for buyers who wanted something else.

2

An architectural bet nobody revisited.

The A380 was a superb answer to a question about air travel that airlines answered differently.

3

A specification waived without a record.

The GM switch went into production known not to meet its own standard.

4

A single point of failure in a critical path.

MCAS trusted one sensor, and 346 people died.

5

Scope that grew while the plan stood still.

Denver added two concourses and twenty airlines without moving a date.

6

A known weakness deferred until the worst possible day.

Southwest knew about SkySolver years before the storm.

THE ANSWER TO THE QUESTION IN THE TITLE

Every one of those six is a topic on this syllabus, and none of them is a coding skill. The engineering is not in the construction — it is in the deciding, the recording, the predicting and the accounting, all of which happen before and around the construction, and none of which the construction can compensate for.

Every failure above was **cheap to prevent and ruinous to correct**, and the gap between those two numbers is where this discipline lives.

**Why this matters more here, not less.**

A team with money can absorb a bad decision by throwing another year at it. A health post serving 6,000 people on intermittent power cannot. Constraint is not the enemy of engineering — it is the reason engineering exists, and the Logan is the proof that designing to a hard constraint beats wishing it away.

PART 3

The discipline, and its vocabulary

5

We now put names to what we have just observed. These terms are the working vocabulary of the next fourteen weeks.

3.1 The problem the discipline was invented to solve

In October 1968 a NATO working group met in Garmisch, Germany, and deliberately chose a provocative title for the conference: **Software Engineering**. The word *engineering* was aspirational, not descriptive. The delegates were reacting to a pattern that had become impossible to ignore: hardware capability was doubling every few years, while the ability to produce reliable software for that hardware was not improving at all. Projects were late, over budget, and defective on delivery. The phrase **software crisis** dates from that meeting.

Nearly sixty years later the crisis has changed shape rather than disappeared. Industry surveys still report that a large fraction of software projects are delivered late, exceed budget, or deliver substantially less than was specified. Southwest's SkySolver is a 2022 instance of a 1968 diagnosis. The techniques in this course are the accumulated professional response to that pattern.

3.2 Programming is not software engineering

Every student in this room can already program. That is a necessary but not sufficient condition for this course. The distinction is not one of skill level; it is one of context. Consider three axes.

AXIS 1 — TIME

PROGRAMMING

The code must work *now*. Correctness is judged today.

SOFTWARE ENGINEERING

The code must keep working for years, under changing requirements, on changing platforms, maintained by people who did not write it.

AXIS 2 — SCALE

PROGRAMMING

SOFTWARE ENGINEERING

One author, one head, one file, thousands of lines.

Many authors, many modules, hundreds of thousands of lines. No single person holds the whole system in mind.

AXIS 3 — CONSEQUENCE

PROGRAMMING

A defect costs the author an afternoon.

SOFTWARE ENGINEERING

A defect costs money, reputation, safety, or lives. It must be traced, reported, prioritised, fixed, tested, and released.

A ONE-SENTENCE FORMULATION

Adapted from Titus Winters at Google: **software engineering is programming integrated over time.** Everything we study this semester — requirements, design, testing, configuration management, process — exists because of that integration.

PART 3, CONTINUED

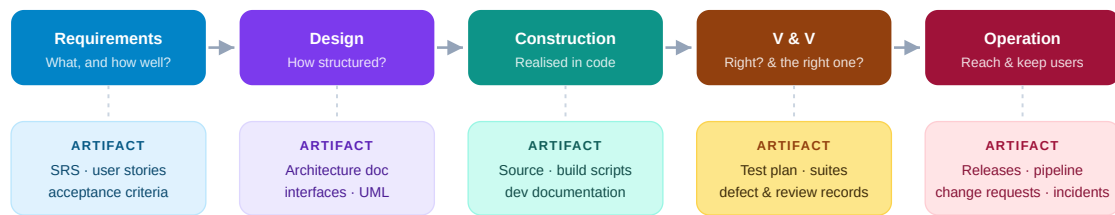
The classical life cycle and its artifacts

Whatever process model a team adopts, the same five kinds of work must be done.

6

The models differ in how the work is ordered, batched, and repeated — not in *whether* it happens. Each phase produces a durable **artifact**, and the artifact is what survives when the people move on.

FIVE PHASES, FIVE ARTIFACTS



The artifact is what survives when the people move on.

T Terminology you will be examined on.

Verification

asks whether the product conforms to its specification.

Validation

asks whether the specification was the right one. A system can pass verification perfectly and still be a failure — the A380, the Edsel and the Nano all did. That is the single most expensive mistake in the discipline, and it is the lower-right quadrant of the diagram above.

3.3 Categories of software, and why the category changes everything



Generic products

Built speculatively and sold to a market the developer defines: Excel, Android, PostgreSQL. **The developer owns the specification.** Change is driven by competitive pressure.



Bespoke (custom) systems

Commissioned by a specific customer: a payroll system for the Ivorian civil service; a patient registry for a district health authority. **The customer owns the specification.** Change is driven by contract negotiation.



Embedded systems

Control physical devices: anti-lock brakes, pacemakers, irrigation controllers. Timing and reliability constraints dominate; deployment is expensive or impossible to repeat.



Data-intensive and AI-enabled

Derive behaviour from data rather than from code alone. Their specification problem is unusually hard, because the intended behaviour cannot be fully written down in advance.

? Ask the category question first.

It determines who has authority over the requirements, how tolerant the system is of defects, and how quickly you can ship a correction. Which category is SANTE-BASSAM? Which category was MCAS? Your answers should differ, and the difference should change how you would work on each.

PART 4

Where the money actually goes

Students consistently underestimate maintenance. Between 60% and 80% of the lifetime cost of a long-lived system is spent *after* first delivery.

That maintenance divides roughly into four buckets, and only one of them is what students expect.

C

Corrective — fixing defects.

Typically the smallest share, around 20% of maintenance. This is the only bucket most students think about.

A

Adaptive — responding to changes in the environment.

A new operating system version, a new tax rate, a new mobile-money API. Nothing about your system changed; the world moved.

P

Perfective — adding or refining functionality because users asked.

Usually the largest share by a wide margin.

V

Preventive — restructuring code to keep future change affordable.

This is what we will later call paying down technical debt. Southwest deferred it for eighteen years.

!

The practical consequence is severe.

A design decision that saves you two days now and costs the maintainers one day per month for five years is a catastrophic decision — even though it looked efficient on the day you made it.

WORKED PRACTICAL EXAMPLE · GUIDED

What a 500-line script really costs

8

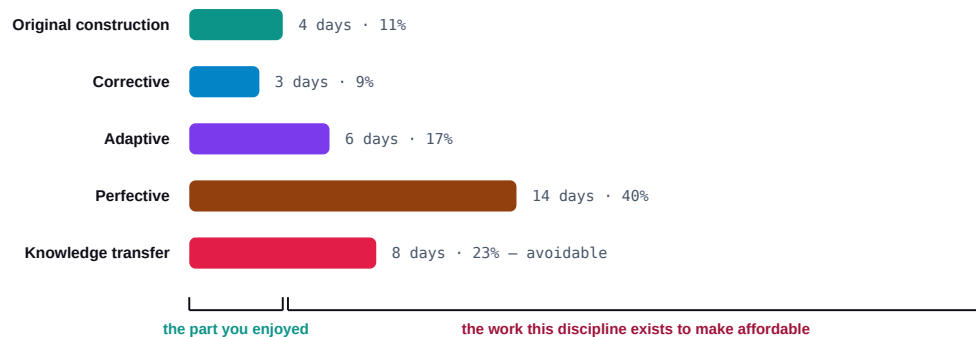
Scenario. A student writes a 500-line Python script that generates end-of-semester grade reports for one department. It works. The department adopts it. Five years later, what did it cost?

We reason in engineer-days, and we count only the work that would not have existed if the script had been discarded after one use.

Bucket	What happened	Engineer-days
Original construction	Initial writing and debugging	4
Corrective	Crash on a student with no middle name; division by zero for a course with no graded work	3
Adaptive	Python 3.9 → 3.12 migration; the registrar changes the CSV export format twice	6

Bucket	What happened	Engineer-days
Perfective	Add GPA computation, then a second grading scale, then PDF output	14
Knowledge transfer	Original author graduates; two successors each reconstruct intent from unreadable code	8
Total	Five-year lifetime cost	35

WHERE THE LIFETIME COST OF A 500-LINE SCRIPT ACTUALLY GOES



Construction is 11% of the total. Everything to its right is what the other twenty-nine sessions of this course are about.

The 8 days of knowledge transfer were *entirely avoidable*: they are the price of having no requirements document, no tests, no README, and no meaningful module structure. Roughly two days of engineering discipline at the start — a short specification, a test file, and named functions with docstrings — would plausibly have removed six of those eight days and reduced the corrective bucket as well.

THE LESSON

Engineering discipline is not overhead imposed on programming. It is an investment whose return is measured over the life of the system — which is exactly why its value is invisible at the moment you must decide to pay for it.

Failure post-mortem

9

Each pair is assigned one case. You may use your phone or laptop to search.

Ariane 5 Flight 501 (1996)

Loss of the launch vehicle 37 seconds after lift-off.

Therac-25 (1985–1987)

Radiation therapy machine that delivered massive overdoses to six patients.

Knight Capital (2012)

Trading firm lost roughly USD 440 million in 45 minutes.

Mars Climate Orbiter (1999)

Spacecraft lost on orbital insertion.

UK Post Office Horizon (1999–2019)

Accounting defects led to the wrongful prosecution of hundreds of sub-postmasters.

Boeing 737 MAX / MCAS

Use the material above, but find one fact this lecture did not state.

Produce three bullet points on a shared document, ready to report to the class in **60 seconds**:

1. State the **proximate technical cause** in one sentence — what the code did wrong.
2. State the **process failure** that let the proximate cause survive to production. Name the life-cycle phase from §3 where it should have been caught.
3. Name **one specific practice** from the course outline that would most plausibly have prevented it.

Expected insight

Do not open this until your pair has written all three bullets.

Reveal

In every one of these cases the proximate technical cause is small and comprehensible — an unhandled conversion, a race condition, a reused flag, a unit mismatch, a single sensor. **The interesting failure is always organisational:** a review that did not happen, a test environment that did not match production, a reused component whose assumptions were never re-validated in the new context, a warning that was logged in the wrong category.

Students who report only the code bug have not yet completed the exercise. That is the same distinction the whole first half of today was making: the code was fine.

TAKE THIS WITH YOU

Six questions before your first line of code

Your team should be able to answer these in writing — not perfectly, but in writing, with names attached, so that in November you can tell whether you were right.

Q1

Who is this for, what do they do today, and what will they stop doing in order to use it?

Name a person, not a category.

Q2

What must be true for this to be worth building?

State the number — patients registered, minutes saved, errors avoided — that would let someone judge you in six months.

Q3

Which constraints are not negotiable?

Power, connectivity, language, literacy, the device in the clerk's hand, the law on health records.

Q4

Which decision, if wrong, is most expensive to reverse later?

That is your architecture. Write it down with its reason, and date it.

Q5

What is the single point of failure?

If one component, one sensor, one person, one file is unavailable — what happens?

Q6

What is the fallback when the clever thing fails?

Denver had none. Your registry will run in a building where the power fails weekly.

None of these can be answered by writing code, and all of them get harder and more expensive to answer the longer you wait. That is the whole argument for designing before implementing, and every case in this lecture is evidence for it.

HOMEWORK AND DELIVERABLES

Due before Session 3 — Tuesday 15 September

11

Submitted to the course GitHub Classroom.

1

Account setup.

Create a GitHub account if you do not have one, accept the CSC 4305 Classroom invitation, and push a file named `about-me.md` to your personal repository containing your name, your programming background, and one software system you use daily that you consider well engineered — with one sentence on why.

2

Reading.

Sommerville, *Software Engineering*, 10th ed., Chapter 1 (or Pressman, Chapter 1). Come to Session 2 able to define the four attributes of good software that Sommerville lists.

3

Written analysis (1 page, PDF, `s01-analysis.pdf`).

Choose a software system you have personally used that failed you — an app that lost your data, a registration portal that rejected valid input, a payment that silently did not go

through. Write: (a) what you as a user observed; (b) your best hypothesis about the underlying defect; (c) which life-cycle phase most plausibly failed; (d) what evidence you would ask for to test your hypothesis if you joined that team tomorrow.

4

Team formation.

Form teams of four (a fifth member only with instructor approval). Post your team name and members to the `#teams` channel by Friday 11 September. Teams that have not self-formed by then will be assigned.



The semester project — SANTE-BASSAM.

From Session 5 onward, every team builds a patient registry and appointment system for a community health post serving roughly 6,000 people, with intermittent electricity and unreliable connectivity. You will take it through requirements, design, implementation, testing, and delivery. Every session from here connects to it — and the six questions above are your first deliverable against it.

REFERENCE

Key terms and further reading

12

KEY TERMS FROM THIS SESSION

software crisis

software life cycle

artifact

verification

validation

design freeze

design-to-cost

generic product

bespoke system

embedded system

data-intensive system

corrective maintenance

adaptive maintenance

perfective maintenance

preventive maintenance

technical debt

single point of failure

traceability

READINGS AND REFERENCES

- Sommerville, I. *Software Engineering*, 10th ed., Pearson, 2016 — Chapter 1.

- Pressman, R. and Maxim, B. *Software Engineering: A Practitioner's Approach*, 9th ed., McGraw-Hill, 2020 — Chapter 1.
- Naur, P. and Randell, B. (eds). *Software Engineering: Report on a Conference Sponsored by the NATO Science Committee*, Garmisch, 1968. Freely available; read the discussion sections, not the papers.
- Leveson, N. and Turner, C. An Investigation of the Therac-25 Accidents. *IEEE Computer* 26(7), 1993.
- Winters, T., Manshreck, T. and Wright, H. *Software Engineering at Google*, O'Reilly, 2020 — Chapter 1.

SOURCES FOR THE CASE MATERIAL

Case	What it documents	Source
Edsel programme and losses	Ford's medium-price venture and its withdrawal after three model years	en.wikipedia.org
Dacia Logan and design-to-cost	Renault Group's own account of the €5,000 brief and the method	renaultgroup.com
Tata Nano end of production	Production and sales collapse, and the positioning problem	caradvice.com.au
GM ignition switch recalls	Timeline, the 2002 specification decision, and the settlement	en.wikipedia.org
Volkswagen emissions	The scale of the payouts and the 2025 convictions	en.ara.cat
A380 wiring and CATIA versions	How two versions of one design tool cost roughly two years	simpleflying.com
A380 versus 787 forecasts	Hub-to-hub against point-to-point, assessed after the fact	centreforaviation.com

Case	What it documents	Source
US DOJ — Boeing 737 MAX charge	The department's statement on what was withheld from the regulator	justice.gov
Denver baggage system	Calleam's reconstruction; the SEBoK entry with scope history	www5.in.tum.de · sebokwiki.org
Southwest 2022 meltdown	The record DOT penalty; SkySolver's batch limits and replacement timeline	itpro.com · simpleflying.com

CSC 4305 — Software Engineering · Session 1 of 28 · Fall 2026

Prof. Guillaume Sanje Mpacko, Ph.D. · School of Science and Engineering · International University of Grand-Bassam

Next · Session 02 — SANTE-BASSAM: The Project, and How It Will Be Delivered · Friday 11 September, 10:00–11:20